

FORCme

FORC Processing Utilities (`forc_funcs.py`)

Functional Documentation, Mathematics, Rationale, and Caveats

Maxwell Brown, Institute for Rock Magnetism, University of Minnesota

April 17, 2026

1 Purpose and scope

This document describes the current design of `forc_funcs.py`, the rationale for the workflow, the mathematical basis of the FORC distribution estimate, and the main practical caveats. The present version supports three user modes through a single notebook-facing entry point:

- **Individual mode** (`mode="i"`): process one FORC file.
- **Stack mode** (`mode="s"`): process multiple repeated FORC files measured with the same acquisition parameters, combine them in magnetization space, and compute a single stacked FORC distribution.
- **Batch mode** (`mode="b"`): process multiple files independently in sequence, returning one output object per file.

2 Program functions

Given one file or a directory of files, the program:

1. Segments the numeric data into calibration points and FORC curves.
2. Optionally corrects drift using repeated calibration points.
3. Optionally replaces the first and/or last points of each FORC using linear extrapolation from interior points.
4. Optionally subtracts a lower branch before ρ calculation.
5. Optionally regrid FORCs in (B, M) space before building the FORC grid.
6. Builds a gridded $M(H_b, H_a)$ array and estimates $\rho(H_b, H_a)$ using a local quadratic LOESS fit.
7. Plots the result in both (H_b, H_a) and transformed (B_u, B_c) space.
8. Exports figures and datafiles.

3 Program outputs

The program outputs .png files of the FORC distribution in Bc,Bu space, and data files and plots of vertical and horizontal profiles. The outputs are automatically named:

- `mode="i"`: the title defaults to the input file name without its extension.
- `mode="b"`: each output uses its own file name without extension.
- `mode="s"`: the title defaults to the common part of the matched file names.

A user-supplied `sample_title` may still be used as an override when desired.

Outputs are organized beneath the input directory:

- `FORC_figures` for main FORC figures.
- `FORC_profiles` for profile text files and profile figure exports.

4 Performance

The dominant cost is computing ρ (the FORC distribution) at every grid cell that contains data. The code is fast because:

- It operates on a *rectangular grid* $M[i,j]$ with contiguous `float64` arrays (good memory locality).
- Neighborhood offsets are precomputed once; each evaluation point loops over a fixed list of offsets.
- The LOESS solve is a fixed-size 6×6 weighted normal equation, solved by an inlined Gaussian elimination routine.
- The full inner loops are compiled with `numba (@njit(cache=True))`, removing Python overhead.

If Numba is not available, the module currently raises an error for the “fast” LOESS function rather than silently falling back to slow Python.

Numba accelerates the most computationally expensive part of the workflow, which is the repeated local quadratic LOESS fits used to estimate the FORC distribution ρ . Numba works by compiling selected numerical Python functions at runtime into machine code, rather than executing them through the standard Python interpreter. This is especially effective for code that contains many nested loops, repeated arithmetic operations, and large array-based calculations. The functions `_tricube_numba`, `_gauss_solve_6x6`, and `loess_rho_from_grid_numba` are “decorated” with `@njit`, so that the local weighted least-squares calculations are carried out in compiled form. This substantially reduces Python-level overhead when looping through every valid (H_b, H_a) grid cell, assembling local neighborhoods, forming weighted quadratic systems, and solving for the mixed-derivative term used to calculate ρ . This makes the LOESS-based FORC calculation much faster on dense grids and stacked datasets, while allowing the code to remain in a relatively clear NumPy/Python form rather than requiring a separate low-level language implementation.

5 FORC mathematics

5.1 FORC distribution definition

Each FORC is measured by saturating the sample, decreasing the field to a reversal field H_b , then measuring magnetization M as the applied field is increased from H_b to H_a with $H_a \geq H_b$. The FORC distribution is

$$\rho(H_b, H_a) = -\frac{1}{2} \frac{\partial^2 M(H_b, H_a)}{\partial H_b \partial H_a}.$$

In practice, M is only known on a discrete, irregular set of points, and must be smoothed before numerical differentiation. This code estimates $\rho(H_b, H_a)$ by fitting a local *quadratic* surface to nearby points and reading off the cross-term coefficient (details below).

5.2 Transformation to (B_u, B_c)

The plotting transformation used is

$$B_u = \frac{H_a + H_b}{2}, \quad B_c = \frac{H_a - H_b}{2}.$$

5.3 Local quadratic LOESS estimate

At each grid node, the code fits a weighted quadratic surface

$$M \approx \beta_0 + \beta_1 \Delta H_a + \beta_2 \Delta H_b + \beta_3 (\Delta H_a)^2 + \beta_4 (\Delta H_a)(\Delta H_b) + \beta_5 (\Delta H_b)^2,$$

and uses

$$\rho(H_b, H_a) = -\frac{1}{2} \beta_4.$$

5.3.1 Weights (tricube kernel)

Neighbors are selected within an ellipse in index space. Each neighbor is assigned a distance proxy $u \in [0, 1]$ and weight

$$w(u) = (1 - u^3)^3, \tag{1}$$

a standard LOESS tricube weight (Cleveland, 1979).

5.3.2 Quadratic Fitting

A quadratic captures local curvature and the cross-term $\Delta H_a \Delta H_b$ needed for the mixed derivative. It is a common choice in FORC smoothing implementations and is a minimal model that supports ρ estimation from local regression rather than differencing noisy data directly.

6 Adaptive smoothing

The current version of the code only allows one version of smoothing, though an update using a VARIFORC-style algorithm is planned. To reduce user guesses of an appropriate smoothing factor, a globally adaptive smoothing approach has been developed. This runs at the dataset level rather than spatially variable cell-by-cell. Grid spacing, fill fraction, and a target effective neighborhood size are used to estimate suitable LOESS spans and minimum-point thresholds, which can then be scaled by the user through `smooth_strength` and `min_pts_strength`.

The LOESS neighborhood is controlled by spans in H_a and H_b (`span_Ha_T`, `span_Hb_T`) and a minimum number of points `min_pts`. The helper `guess_loess_params`:

1. Estimates grid steps dH_a and dH_b from medians of differences.
2. Computes the *fill fraction* $f = N_{\text{finite}}/N_{\text{total}}$ of the gridded M array.
3. For a candidate ellipse (rx, ry) in index units, counts offsets inside the ellipse (N_{cand}) and estimates the expected number of usable points as $N_{\text{eff}} \approx f N_{\text{cand}}$.
4. Grows (rx, ry) until N_{eff} meets a target (default `target_n_eff=60`), subject to caps (`max_rx`, `max_ry`).
5. Sets `min_pts` to a fraction of N_{eff} (default `min_pts_frac=0.55`) with floors/ceilings.

This approach tries to keep the local regression well-determined (enough points), while not over-smoothing. Although subjective, results should be assessed and smoothing parameters adjusted as necessary (as one would do manually in a number of other FORC software).

7 Display and plotting design

The (B_u, B_c) plot uses rotated *cell edges* for the color image and rotated *cell centers* for contours. This avoids the coercivity-axis offset that arises when a rotated center grid is passed directly to `pcolormesh`. A plotting-only upsampling option remains available, controlled from the notebook through `display_upsample_factor`. The default is 0. This is the rawest representation of the data and can look blocky if a coarse H_u increment was chosen for the FORC measurement. Upsampling increases in 2s, i.e. 0, 2, 4, etc. 2 is often sufficient. Note that there is a trade off between the degree of upsampling and the data area. Increased upsampling reduces the data area, which is most noticeable near $B_u = 0$.

8 Mode-specific behavior

8.1 Individual mode

Individual accepts and processes one input file.

8.2 Stack mode

Stack mode is intended for repeated FORC measurements made on the same specimen with the same acquisition parameters. Each input file is prepared independently, mapped onto a common $M(H_b, H_a)$ grid, and then combined using a user-selected stack statistic before the final LOESS estimate of ρ .

8.3 Batch mode

Batch mode processes many files independently. It returns one output dictionary per input file. Automatic and custom profiles are then generated afterward as separate notebook steps.

9 Key caveats

- The file parser assumes comma-separated field–moment pairs and expected header tags.
- Calibration separation relies on identifying the leading point of a block near HCal.
- Sparse regions or too-large `min_pts` values may leave many ρ cells undefined.
- Although the code aims to minimize boundary effects, they may appear near $H_a \approx H_b$ or at the ends of FORCs.
- Normalization is shown for plotting rather than calculated ρ units. This choice was made to expedite the users understanding of scale variations on the FORC diagrams and profiles. `normalize_to_unit=True` scales by a percentile of $|\rho|$; values above that percentile clip to ± 1 in the colormap. This affects only the visualization, not the underlying ρ values.
- Plotting-only upsampling is for visualization only.